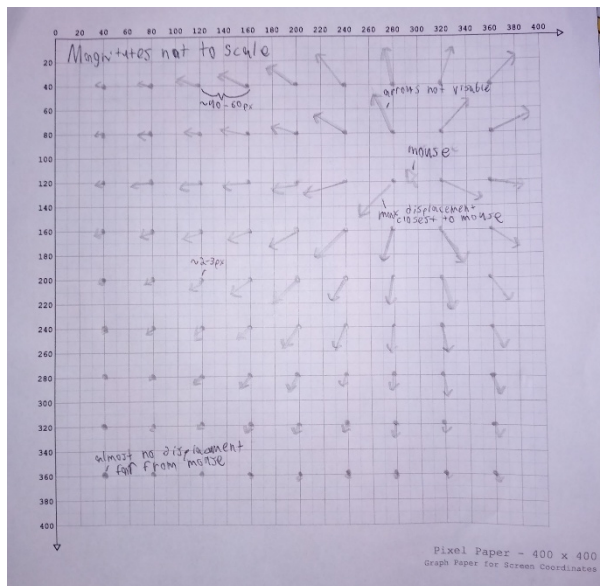


Idea 1: Repelled dots

A grid of black dots (probably circles with small radius) that are pushed away from the mouse as it moves.

- The dots have fixed 'base' positions
- The positions are changed based on proximity to the mouse
 - Compare mouse coordinates to the dots'
- Dots should 'move' directly away from the mouse: meaning they should be on a straight line that passes through their starting positions and the mouse (may be difficult; may require vector math)
- The distance they move from the base positions should increase as the mouse gets closer and drop off as it gets farther
 - Try different functions: linear, x^{-1} , $x^{1/2}$, $\log(x)$
- If updated every frame, the dots should (hopefully) appear to move smoothly
- Different approaches (alternative measures used for the calculations) may be required to make it look nice



Idea 2: Pizza that gets eaten

A drawing of a pizza that gets 'eaten' when the user presses a button or mouse (many different options from implementation).

- Pizza made from circle (fill cheese) with outline (crust) and smaller circles for pepperoni
 - Or made of arcs (slices); arc outlines would appear in middle so there would have to be 2 layers of arcs for cheese and crust

- Bite marks: 3 circles or arcs which are the same colour as the background that cover a portion of the pizza, making it disappear
- Two eating options; implement 1 or both (different buttons? Alternate?)
 - Whole: the bite marks appear on the whole pizza, going from one side to the other as the button is pressed (create new ones so the rest of the pizza stays covered)
 - Slices: separate pizza into slices (with gaps) and 'eat' them one at a time in sequence
 - Do not combine; when one way is started, stick to it until the pizza is gone
- Potentially replace pizza
 - Pop in? slide in? grow from point?
 - Make sure to remove all the previous bite marks and/or pizza
 - § Easiest is to just remove bite marks, revealing the pizza underneath

Idea 3: Eyes

A pair of eyes that follow the mouse and are agitated when clicked.

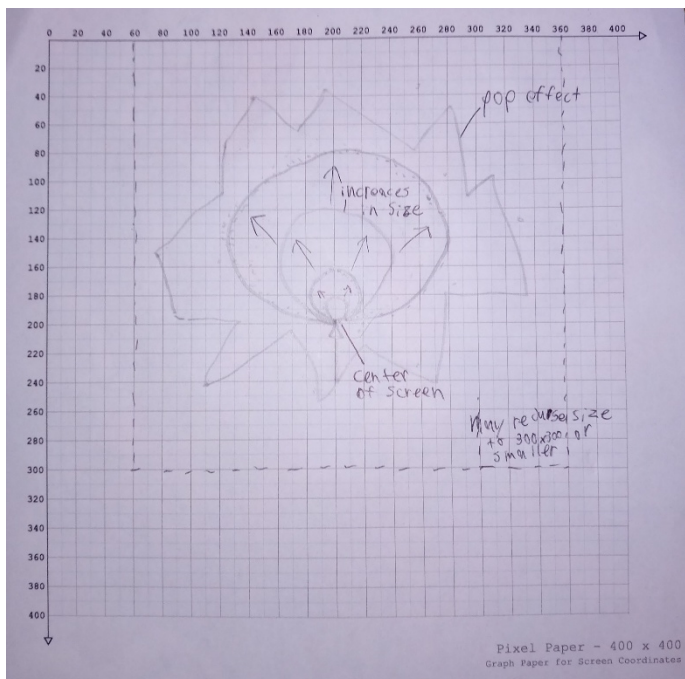
- The pupils and irises are just circles. They are surrounded by curved lines (method unknown) that form an eye shape
- The pupils will be displaced from their center positions towards the mouse by a slightly varying amount based on proximity to the mouse (increased as mouse gets further), with a cap
 - Try different functions
- If part of a pupil would go outside the boundary created by the curved lines, that part should be covered (method unknown)
- If the mouse is hovering over an eye and clicks, the white of the eye will turn slightly pink and the eye will appear to squint
 - These effects will slowly fade
 - If an eye is clicked again before the effects fully fade, they will worsen, until the eye is red; make sure the eye doesn't fully close
 - Bonus: tears could be easily made with arcs and triangles

Idea 4: Fillable balloon

A balloon that is filled by user input until it pops.

- the balloon starts out as a small red circle that gets bigger when the user scrolls up with the scroll wheel (or pushes a button)
 - The balloon would have a string like the one in the challenge

- o Possibly have it change colour
- ALTERNATIVE: instead of a circle that gets bigger, the balloon could start as a triangle, and become higher sided polygons
- When it reaches a certain threshold (preset or random), it pops
 - o Create a big pop effect using lines; potentially with “pop” text
 - o Bonus: animate the string and/or remnants of the balloon to fall
- Potentially add ability to deflate the balloon by scrolling down
 - o Add a shape to show air escaping
- Bonus: add a special button that lets go of the balloon, so it flies away
- Instead of inflating in discrete steps, it could be animated to grow smoothly
 - o Easier with buttons; just continuously increase the size while the button is held down
 - o For scroll wheel: would need to store amount to grow in a variable, and decrease it every frame



Idea 5: Spinning rings

A series of rings wrapped around each other. They have gaps in them, spin, and are influenced by the user.

- Each ring is an arc between 270 and 360 degrees with a circle the same colour as the background covering the inside
- Smaller rings are layered above larger ones. All rings are aligned to the center

- All the rings spin by changing their start and stop angles every frame. spin rates could be stored in variables as 'degrees/frame'; this would not need to be scaled for larger rings (additional variables would have to store the current rotations)
- Various buttons and/or mouse clicks can change the spin rate and/or direction of individual rings to make them out of sync (many different possible implementations)
 - o One possibility: fewer buttons; one ring is selected at a time (highlighted), only that ring can be modified, but the user can 'scroll' between the rings